Experiment 1

Implement a trie data structure to efficiently support autocomplete suggestions based on user input in google docs.

Aim:

To implement a trie data structure to efficiently support autocomplete suggestions based on user input.

Algorithm:

1. Initialize Trie

- Create a root node which acts as the starting point of the trie.

2. Insert a word into the Trie

- Start from the root node.
- For each character in the word:
    - If the character is not present in the current node's children:
        - Add the character to the children with a new TrieNode.
    - Move to the child node of the character.
- After the last character, mark isEndOfWord as True.

3. Search for a Prefix

- Start from the root node.
- For each character in the prefix:
    - Check if the character exists in the current node's children.
    - If not found, return False (or Null).
    - Move to the child node of the character.
- Return the node if the end of the prefix is reached.

4. Collect Words Starting from a Node

- If the current node marks the end of a word, add the prefix to the result list.
- For each child node:
    - Recursively collect words starting from the child node, appending the current character to the prefix.

5. Autocomplete Function

- Use the Search for a Prefix function to find the node corresponding to the last character of the input prefix.
- If the prefix node is found, use the Collect Words function to gather all words beginning with the prefix.
- Return the list of words.

Program:

```
class TrieNode:
        def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
        def __init__(self):
        self.root = TrieNode()

        def insert(self, word):
        node = self.root
        for char in word:
        if char not in node.children:
                node.children[char] = TrieNode()
        node = node.children[char]
        node.is_end_of_word = True

        def search(self, prefix):
        node = self.root
          for char in prefix:
        if char not in node.children:
                return []
        node = node.children[char]
        return self._find_words_from_node(node, prefix)

        def _find_words_from_node(self, node, prefix):
        words = []
        if node.is_end_of_word:
        words.append(prefix)

        for char, next_node in node.children.items():
        words.extend(self._find_words_from_node(next_node, prefix + char))

        return words
```

```
def main():
    trie = Trie()

    while True:
    print("Choose an option:")
    print("1. Insert word")
    print("2. Get autocomplete suggestions")
    print("3. Exit")

    choice = input("Enter your choice: ")

    if choice == '1':
    word = input("Enter the word to insert: ").strip()
    trie.insert(word)
    print(f"Inserted '{word}' into the trie.\n")

    elif choice == '2':
    prefix = input("Enter the prefix to search: ").strip()
    suggestions = trie.search(prefix)
    if suggestions:
        print(f"Autocomplete suggestions for '{prefix}': {suggestions}\n")
    else:
        print(f"No suggestions found for '{prefix}'.\n")

    elif choice == '3':
    print("Exiting...")
    break

    else:
    print("Invalid choice. Please choose again.\n")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
Output
Choose an option:
1. Insert word
2. Get autocomplete suggestions
3. Exit
Enter your choice: 1
Enter the word to insert: apple
Inserted 'apple' into the trie.

Choose an option:
1. Insert word
2. Get autocomplete suggestions
3. Exit
Enter your choice: 1
Enter the word to insert: application
Inserted 'application' into the trie.

Choose an option:
1. Insert word
2. Get autocomplete suggestions
3. Exit
Enter your choice: apricot
Invalid choice. Please choose again.

Choose an option:
1. Insert word
2. Get autocomplete suggestions
3. Exit
Enter your choice: 2
Enter the prefix to search: ap
Autocomplete suggestions for 'ap': ['apple', 'application']

Choose an option:
1. Insert word
2. Get autocomplete suggestions
3. Exit
Enter your choice: 3
Exiting...
```

Result:

Thus the program has been implemented Successfully.